INTERN PROJECT BRIEF

Reading Tracker

A JavaScript fundamentals project — built around something you love.

Assigned by: Senior Developer · Stack: HTML · CSS · Vanilla JS · Est. time: 2–3 weeks

Difficulty: Beginner–Intermediate · Revision: 1.0 · For internal use only

1. Project Overview

Welcome to your first solo project! This reading tracker will let you add books, track your reading status, write personal notes, and view your progress — all without a backend or database. Everything lives in the browser using **localStorage**.

> *Senior Dev Note: Real-world projects always start with a brief like this. Before writing a single line of code, you should be able to answer: What does this do? Who is it for? What does done look like? Read this document fully before opening your editor.*

Why This Project?

- It covers the core JS concepts every frontend role will test you on.
- It's small enough to finish, but rich enough to put on a portfolio.
- You'll build something you can actually use every day.

2. Functional Requirements

These are the features your finished app must have. Think of these like acceptance criteria in a real ticket — the feature is not done until all criteria are met.

Feature	JS Concept	What You'll Learn
Add a Book	DOM + Events	A form with Title, Author, Genre, and Status fields. On submit, the book appears in the list without a page reload.

Feature	JS Concept	What You'll Learn
Book Status	Conditionals	Each book has one of three statuses: Want to Read, Currently Reading, or Finished. Status is colour-coded.
Edit a Book	Array methods	Click any book to edit its details in-place. Changes persist after page refresh.
Delete a Book	splice / filter	A delete button removes the book from the list and from storage. A confirmation prompt prevents accidents.
Search / Filter	Array filter + input events	A search bar filters the visible list in real-time by title or author as you type.
Star Rating	Loops + DOM	After finishing a book, assign it 1–5 stars. Render stars visually (e.g. ★★★■■).
Personal Notes	Template literals	A textarea on each book card for private thoughts. Saved automatically on blur.
Reading Stats	reduce + math	A summary panel showing: total books, books finished, favourite genre, and average rating.
Persist Data	localStorage	All books survive a page refresh. Use JSON.stringify / JSON.parse to store and retrieve.
Sort Books	Array sort	Sort the list by title (A–Z), rating (high–low), or date added (newest first).

> Stretch Goals (optional): Add a 'currently reading' progress bar (page X of Y). Export your list as a .csv file. Add a dark mode toggle using CSS custom properties.

3. Suggested File Structure

Keep things simple. Three files is all you need for this project. Resist the urge to over-engineer — that's a trap even senior devs fall into.

File	Purpose
`index.html`	The only HTML page. Contains the form, book list container, and stats panel.
`style.css`	All visual styling. Use CSS custom properties (variables) for colours.
`app.js`	All JavaScript. No frameworks, no libraries. 100% vanilla JS.

> *Senior Dev Note: One JS file feels small now — and it is. But the discipline of keeping HTML structure, CSS presentation, and JS behaviour separated is a professional habit worth building from day one. Don't mix them.*

4. JavaScript Concepts You'll Use

Every concept below will come up in a real frontend interview. Each one is introduced naturally as you build the features above.

Feature	JS Concept	What You'll Learn
querySelector / querySelectorAll	DOM Access	Selecting elements to read or modify. You'll use these constantly.
addEventListener	Events	Listening for clicks, input changes, form submissions, and blur events.
Array.push / filter / find / sort	Array Methods	The bread and butter of working with lists of data in JavaScript.
Array.reduce	Aggregation	Used in your stats panel to count totals, sum ratings, find the most common genre.
localStorage	Web Storage API	Saving and loading your book list across page refreshes using the browser.
JSON.stringify / JSON.parse	Serialisation	Converting your JS objects to a string for storage, and back again.
Template Literals	String Syntax	Writing HTML strings cleanly using backticks and ${ } expressions.
Arrow Functions	ES6 Syntax	The modern, concise way to write functions — used everywhere in real codebases.

Feature	JS Concept	What You'll Learn
Ternary Operator	Conditionals	Inline if/else — used for status colours, star rendering, and conditional text.
Event Delegation	Performance	One listener on a parent instead of many on children. Important for dynamic lists.

5. Build Milestones

Break the project into phases. Finish one phase completely before starting the next. This is how real sprints work — partial progress on everything is worse than full progress on something.

Phase	Goal	Deliverable / Done When...
1	**HTML Skeleton**	index.html has a working form and an empty list container. Open in browser, form is visible.
2	**Add + Display**	Submitting the form appends a book card to the page. No persistence yet — page refresh clears it.
3	**Persist to Storage**	Books survive a page refresh. console.log(localStorage) shows your data after adding a book.
4	**Delete + Edit**	Each card has working delete (with confirm) and edit buttons. Changes persist.
5	**Search + Sort**	Typing in the search bar filters cards live. Sort dropdown reorders the list correctly.
6	**Ratings + Notes**	Star rating updates and saves on click. Notes textarea auto-saves on blur.
7	**Stats Panel**	Summary shows accurate totals, average rating, and top genre using reduce.
8	**Polish + Deploy**	Looks good on mobile. No console errors. Hosted on GitHub Pages and shareable.

> Commit after every phase. Your commit history is part of your portfolio — a project with one commit looks like you built it in an afternoon. Aim for at least 15–20 meaningful commits with clear messages (e.g. 'feat: add delete with confirm dialog').

6. Resources for When You're Stuck

Being stuck is normal. The difference between a junior and a senior is knowing where to look and how long to sit with a problem before asking for help. Rule of thumb: try for 20–30 minutes first, then reach out.

Core Reference

Resource	Type	Best For
MDN Web Docs (developer.mozilla.org)	Reference	The gold standard. For any JS method, event, or API — check MDN first. Not w3schools.

Resource	Type	Best For
javascript.info	Tutorial	The best free deep-dive into JavaScript. Read the first three chapters before starting.
CSS-Tricks	Reference	For flexbox, grid, and CSS custom properties. Has the definitive flexbox guide.
The Odin Project	Curriculum	Free, project-based curriculum. Their JS Foundations module parallels this project exactly.

When You Hit a Specific Wall

Resource	Type	Best For
MDN: localStorage	Reference	Search 'MDN localStorage' — the Using the Web Storage API guide is the best starting point.
MDN: Array methods	Reference	Search 'MDN Array.prototype.filter' (or reduce, find, sort). Read the examples section.
MDN: addEventListener	Reference	Covers event types, the event object, and bubbling — all relevant to this project.
MDN: Event Delegation	Guide	Search 'MDN event delegation' — essential for your dynamic book list.
JSON MDN	Reference	Search 'MDN JSON.stringify' and 'MDN JSON.parse'. Understand the round-trip.

> *How to Google like a dev: Instead of 'how do I make a list in JavaScript', search 'MDN Array push' or 'javascript add item to array'. Be specific. Include the method name if you know it, or the outcome if you don't.*

7. Real-World Dev Habits to Practice

The code is only half the job. These habits separate developers who grow quickly from those who plateau.

Use Git from day one	Create a GitHub repo before writing a line of code. Commit every time something works. Your commit history tells a story.
Read errors out loud	Before Googling an error, read the full message. The filename and line number are right there. Train yourself to look.
Console.log everything	It's not cheating — every developer does it. Log your data before and after you transform it. It makes bugs obvious.
Comment your 'why'	Don't comment what the code does (it should be readable). Comment why you made a decision, or what a tricky block is solving.
Build in the browser	Open DevTools (F12). Keep the Console tab open. Refresh constantly. See the result before writing more code.
One feature at a time	Close all other features mentally. If you're building delete, don't touch the search bar. Context switching causes bugs.
Name things clearly	bookList not bl. addBookToStorage not abs. You'll thank yourself in two weeks when you come back to read the code.
Don't copy-paste blindly	If you find a Stack Overflow answer, type it out manually. The act of typing forces you to understand each part.

8. Definition of Done

Your project is complete when ALL of the following are true. Be honest with yourself — partial credit doesn't exist in production.

- ■ All 10 features from Section 2 are working
- ■ Data persists after closing and reopening the browser tab
- ■ No errors in the DevTools Console
- ■ The app works on a mobile screen (use Chrome DevTools device emulator)
- ■ Code is on GitHub with a descriptive README explaining what the app does
- ■ Live demo is deployed on GitHub Pages
- ■ At least 15 commits with meaningful messages
- ■ Someone else can use it without you explaining anything

Portfolio tip: Screenshot the finished app, write 3 sentences about what you built and what you learned, and pin the GitHub repo. This is your first portfolio piece. Own it.

A note from your senior dev

Everyone builds a to-do list or calculator as their first project. You're building something personal — a reading tracker you'll actually want to open. That matters. When you care about what you're building, you push through the stuck moments instead of abandoning it. You're going to hit walls. The localStorage data will corrupt. A button won't work and you won't know why. You'll delete a feature by accident. This is all normal. It's the job. When that happens: read the error, console.log your data, check MDN, and ask for help if you're still stuck after 30 minutes. That's the process. Every time. Good luck — see you at code review.